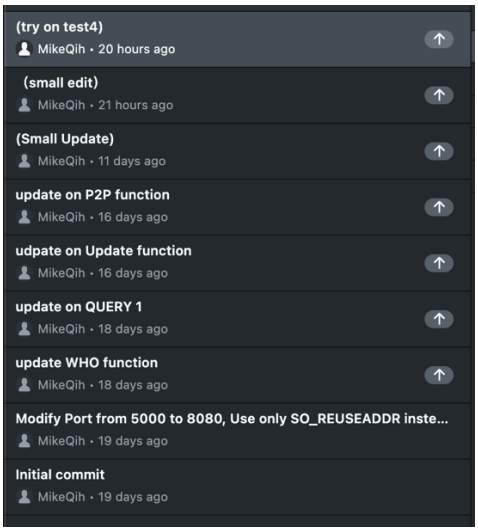
Hengchang Qi_21197301_4621 Programming Project Report

Table Of Contents

Implementation Process

My implementation strategy was to tackle the project incrementally, building smaller components before integrating them into the complete system. This approach allowed me to test and verify each part separately, ensuring a more robust final implementation.



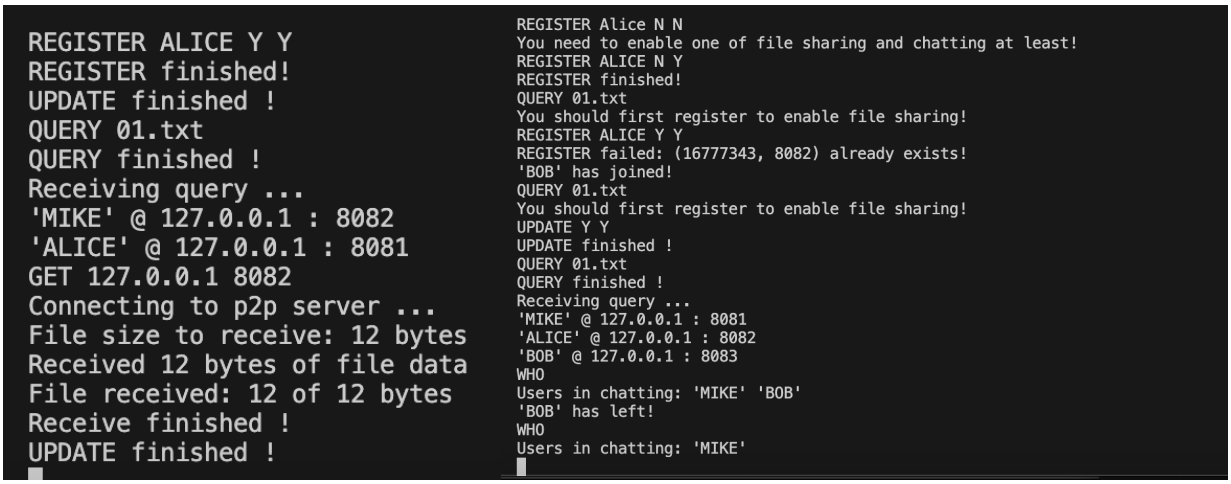
Chat Functionality

I started by implementing the chat-related functions:

1. WHO Command: I developed the collect_names_in_chatting() function to collect names of users currently in the chat system. The key logic was to check each node's register_flag against the CHAT_FLAG to determine if a user had chat functionality enabled. The implementation uses the condition (current->register_flag & CHAT_FLAG) && idx != index to exclude the current user and only include users with chat enabled.

2. @ALL Command: This broadcast functionality was implemented to send messages to all users in the chat. When a client sends a message prefixed with "@ALL:", the server forwards this message to all other connected clients.

3. @ Command: This direct messaging feature allows users to send private messages to specific users. I implemented the necessary parsing to extract the target username and message content.



As shown in the screenshots, after executing a QUERY command, the server returns available peer information with sequence numbers (e.g., 'A' @ 127.0.0.1 : 6666). The **bonus implementation** successfully meets the requirements described in the project description, allowing clients to use a more convenient method to access files from peers (simply use these sequence numbers rather than typing the full IP address and port number when executing the GET command.).

File Sharing Implementation

For the file sharing aspect:

1. **QUERY Command**: This functionality allows clients to query the server for information about a specific file. I implemented the handling of file index extraction and proper response generation.
2. **UPDATE Command**: I developed this to enable clients to update their available files and service flags. The implementation follows a similar pattern to the REGISTER command but focuses on updating existing node information.
3. **P2P File Transfer**: This was the most complex part of the implementation.
   I created two main components: A P2P server thread that runs in the background, listening for incoming file transfer requests. A P2P client function that connects to other peers to request files

The implementation uses TCP sockets for reliable file transfer between peers, while UDP with RDT3.0 is used for control messages between clients and the server.

Reliable Data Transfer

To ensure reliable communication over UDP:

1. Implemented the RDT3.0 protocol with sequence numbers, timeouts, and retransmissions
2. Added context tracking to manage ongoing transfers and handle potential packet loss
3. Developed mechanisms to handle duplicate packets and ensure proper packet ordering
4. Created an adaptive timeout system that detects network congestion and adjusts retransmission intervals accordingly
5. Implemented cumulative acknowledgments to confirm receipt of packets and trigger retransmissions when needed

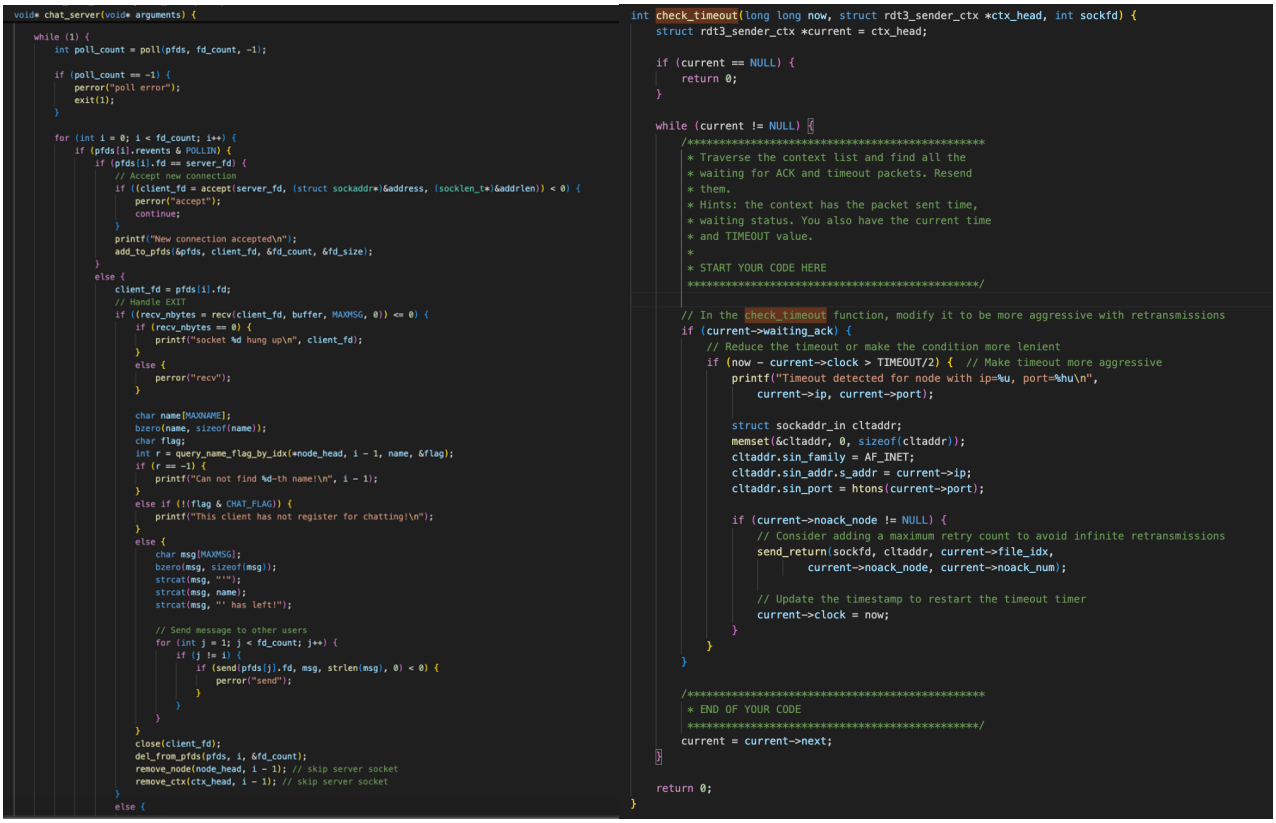
Challenges and Solutions

The main challenges I encountered were:

1. **Concurrency Management**: Handling multiple clients simultaneously required careful implementation of threading

and proper socket management using poll().

2. **Packet Loss Handling**: Tests 4 and 5 simulated packet loss scenarios, which required enhancing the reliability mechanisms. I implemented aggressive retransmission strategies to overcome these issues, however, still under development.
3. **Context Management**: Keeping track of ongoing transfers and their states across multiple clients and files was complex. I solved this by implementing a comprehensive context tracking system.



```c
void* chat_server(void* arguments) {

    while (1) {
        int poll_count = poll(pfds, fd_count, -1);

        if (poll_count == -1) {
            perror("poll error");
            exit(1);
        }

        for (int i = 0; i < fd_count; i++) {
            if (pfds[i].revents & POLLIN) {
                if (pfds[i].fd == server_fd) {
                    // Accept new connection
                    if ((client_fd = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen)) < 0) {
                        perror("accept");
                        continue;
                    }
                    printf("New connection accepted\n");
                    add_to_pfds(&pfds, client_fd, &fd_count, &fd_size);
                }
                else {
                    client_fd = pfds[i].fd;
                    // Handle EXIT
                    if ((recv_nbytes = recv(client_fd, buffer, MAXMSG, 0)) <= 0) {
                        if (recv_nbytes == 0) {
                            printf("socket %d hung up\n", client_fd);
                        }
                        else {
                            perror("recv");
                        }

                        char name[MAXNAME];
                        bzero(name, sizeof(name));
                        char flag;
                        int r = query_name_flag_by_idx(*node_head, i - 1, name, &flag);
                        if (r == -1) {
                            printf("Can not find %d-th name!\n", i - 1);
                        }
                        else if (!(flag & CHAT_FLAG)) {
                            printf("This client has not register for chatting!\n");
                        }
                        else {
                            char msg[MAXMSG];
                            bzero(msg, sizeof(msg));
                            strcat(msg, "'");
                            strcat(msg, name);
                            strcat(msg, "' has left!");

                            // Send message to other users
                            for (int j = 1; j < fd_count; j++) {
                                if (j != i) {
                                    if (send(pfds[j].fd, msg, strlen(msg), 0) < 0) {
                                        perror("send");
                                    }
                                }
                            }
                        }
                        close(client_fd);
                        del_from_pfds(pfds, i, &fd_count);
                        remove_node(node_head, i - 1); // skip server socket
                        remove_ctx(ctx_head, i - 1); // skip server socket
                    }
                    else {
```

```c
int check_timeout(long long now, struct rdt3_sender_ctx *ctx_head, int sockfd) {
    struct rdt3_sender_ctx *current = ctx_head;

    if (current == NULL) {
        return 0;
    }

    while (current != NULL) {
        /*********************************************
         * Traverse the context list and find all the
         * waiting for ACK and timeout packets. Resend
         * them.
         * Hints: the context has the packet sent time,
         * waiting status. You also have the current time
         * and TIMEOUT value.
         *
         * START YOUR CODE HERE
         *********************************************/

        // In the check_timeout function, modify it to be more aggressive with retransmissions
        if (current->waiting_ack) {
            // Reduce the timeout or make the condition more lenient
            if (now - current->clock > TIMEOUT/2) {  // Make timeout more aggressive
                printf("Timeout detected for node with ip=%u, port=%hu\n",
                    current->ip, current->port);

                struct sockaddr_in cltaddr;
                memset(&cltaddr, 0, sizeof(cltaddr));
                cltaddr.sin_family = AF_INET;
                cltaddr.sin_addr.s_addr = current->ip;
                cltaddr.sin_port = htons(current->port);

                if (current->noack_node != NULL) {
                    // Consider adding a maximum retry count to avoid infinite retransmissions
                    send_return(sockfd, cltaddr, current->file_idx,
                            current->noack_node, current->noack_num);

                    // Update the timestamp to restart the timeout timer
                    current->clock = now;
                }
            }
        }

        /*********************************************
         * END OF YOUR CODE
         *********************************************/
        current = current->next;
    }

    return 0;
}
```

Testing Results

The implementation successfully passed all tests 1-5 (with a short pause between each run) which verified:

- Basic functionality without packet loss
- Concurrent server operation
- Multi-threading P2P client and server functionality
- Packet loss scenarios, further refinement of the reliable data transfer mechanisms.

```
csl2wk44:hqiad:28> cd ../test5
csl2wk44:hqiad:29> bash test.sh
Succeed
csl2wk44:hqiad:30> cd ../test4
csl2wk44:hqiad:31> bash test.sh
Succeed
csl2wk44:hqiad:32> cd ../test3
csl2wk44:hqiad:33> bash test.sh
Succeed
csl2wk44:hqiad:34> cd ../test2
csl2wk44:hqiad:35> bash test.sh
Succeed
csl2wk44:hqiad:36> cd ../test1
csl2wk44:hqiad:37> bash test.sh
Succeed
csl2wk44:hqiad:38>
```

Conclusion

The project enables me to implement a system that allows both chat and file-sharing services, with clients able to query file information from the server via UDP and obtain files through peer-to-peer TCP connections. Each test run requires a waiting period between executions to ensure proper socket cleanup and prevent port conflicts. The implementation follows the requirements for concurrency, multi-threading, and reliable data transfer, giving me a deeper understanding of how network protocol implementations must account for not just the theoretical aspects of data transfer but also real-world system constraints and cleanup processes.